

Git Branch Overview

Class no:2

Git Branch

A **Git branch** is an independent line of development that allows you to work on new features, bug fixes, or experiments without affecting the main codebase. Technically, a branch is a **lightweight pointer to a specific commit** in the repository's history, enabling isolated changes until they are ready to be merged back.

Branches are central to Git's workflow because they make it easy to:

- **Isolate work** for features, fixes, or experiments.
- **Collaborate** without interfering with others' changes.
- **Maintain stability** in the main branch while development continues elsewhere.

Common Branch Types include:

- **Main/Master** – Stable, production-ready code.
- **Feature** – For developing specific features.
- **Hotfix** – For urgent production fixes.
- **Release** – For preparing a new version.
- **Develop** – Integration branch for ongoing work.

Basic Commands:

List all local branches → **git branch**

Create a new branch → **git branch <branch-name>**

Switch to an existing branch → **git switch <branch-name>**

Create and switch to a new branch → **git switch -c <branch-name>**

Delete a branch (safe) → **git branch -d <branch-name>**

Force delete a branch → **git branch -D <branch-name>**

Rename a branch → **git branch -m <new-branch-name>**

Hands on creating a repo with multiple branches

Step no:1 Create a remote repo in github (Keep it public and enable Readme file).

1 General

Owner *  NITHYAVEL04 / Repository name * myrepo1

✔ myrepo1 is available.

Great repository names are short and memorable. How about [ubiquitous-memory?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *  Public

Choose who can see and commit to this repository

Add README On ☒

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license

Licenses explain how others can use your code. [About licenses](#)

Create repository

Step no:2 Copy the url and clone it in gitbash.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~  
$ cd downloads  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads  
$ git clone https://github.com/NITHYAVEL04/myrepo1.git  
Cloning into 'myrepo1'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.
```

Get inside of our repo

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads  
$ cd myrepo1/  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)  
$ |
```

Create a new file inside it.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads  
$ cd myrepo1/  
  
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)  
$ vi myfile1.com
```

Step no:3 To see the available branches: **git branch**

The symbol * denotes the current branch.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git branch
* main
```

Step no:4 We are going to create a new branch called “dev”

git branch dev

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git branch dev
```

To switch to the dev branch: **git switch dev**

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git switch dev
Switched to branch 'dev'

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ |
```

Create a new file in dev branch

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ touch denbranchfile1.py
```

Add the file and make commit. Name the commit as “First”.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ git add denbranchfile1.py

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ git commit -m "First"
[dev ad7e99a] First
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 denbranchfile1.py
```

Step no:5 Now we are going to see how the main branch and the dev branch maintain the commit history.

The dev shows all the commit history including main as it is created from main.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ git log --oneline
ad7e99a (HEAD -> dev) First
cf80d01 (origin/main, origin/HEAD, main) Initial commit
```

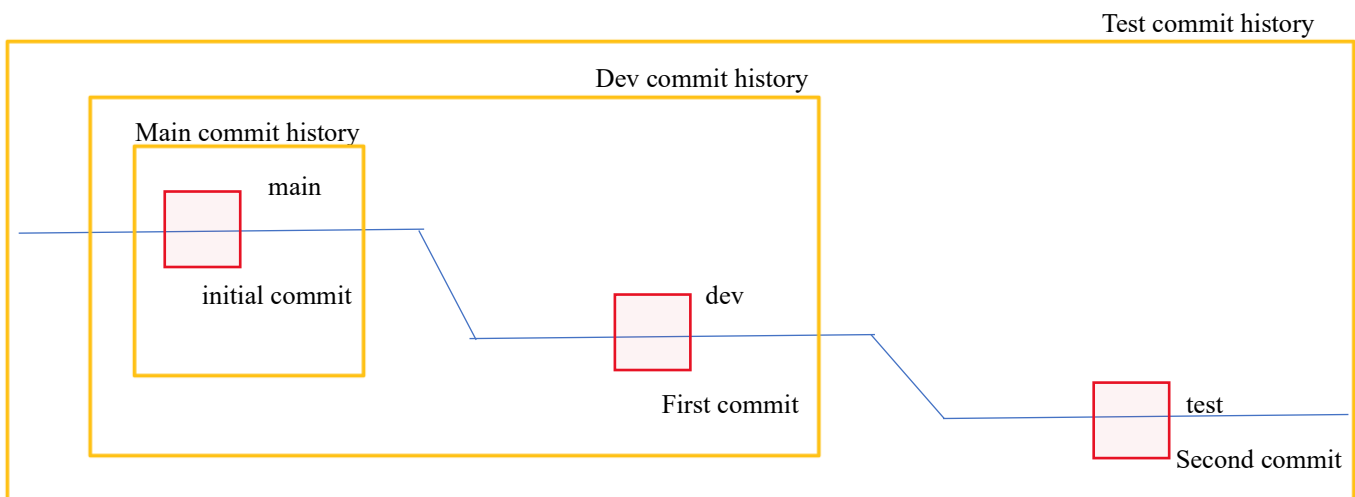
Now get back to the main branch and check the commit history.

The main only show the initial commit, it won't show the commit in the branches created from it.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git log --oneline
cf80d01 (HEAD -> main, origin/main, origin/HEAD) Initial commit
```

The flow can be,

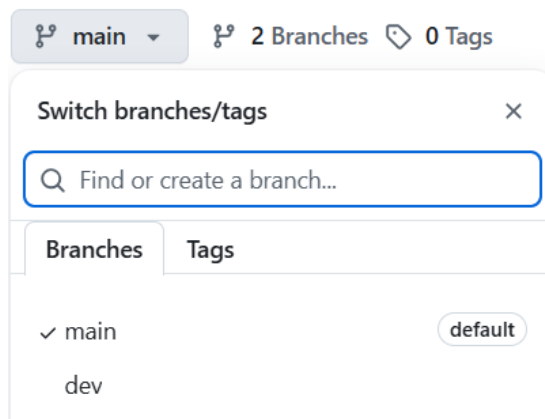


Step no:6 Now we are going to add the dev branch to the remote repo. Checkout to dev branch and push it to remote repo

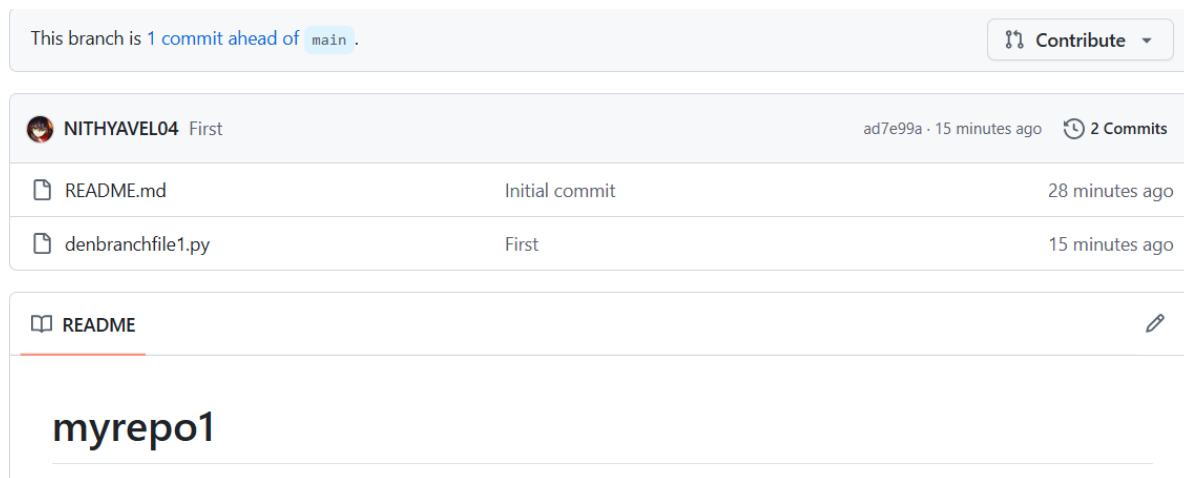
```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git checkout dev
Switched to branch 'dev'

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ git push origin dev
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
```

If we go to GitHub, we can see all the branches has been visible there.



Inside the dev branch we can see the both commits.



Step no:7 We are going to create a new branch called “test” from dev.

git branch test

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ git branch test
```

Switch to test branch.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (dev)
$ git switch test
Switched to branch 'test'

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (test)
$
```

Create a new file inside the test branch.

Add and commit it. Name the commit as “Second”.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (test)
$ touch testfile1.c

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (test)
$ git add .

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (test)
$ git commit -m "Second"
[test f4e8d52] Second
3 files changed, 1 insertion(+)
create mode 100644 myfile1.com
create mode 100644 testfile.java
create mode 100644 testfile1.c
```

Then push them to remote repo.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (test)
$ git push origin test
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 348 bytes | 87.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'test' on GitHub by visiting:
remote:   https://github.com/NITHYAVEL04/myrepo1/pull/new/test
remote:
To https://github.com/NITHYAVEL04/myrepo1.git
 * [new branch]      test -> test
```

If we check the commit history from test, we can see the commits made in main and dev along with test.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (test)
$ git log --oneline
f4e8d52 (HEAD -> test, origin/test) Second
ad7e99a (origin/dev, dev) First
cf80d01 (origin/main, origin/HEAD, main) Initial commit
```

Step no:8 We are going to create a new branch called “support” from test.

git branch support

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (test)
$ git branch support
```

Switch to test branch.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (test)
$ git switch support
Switched to branch 'support'

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (support)
$
```

Create a new file inside the support branch.

Add and commit it. Name the commit as "Third".

Then push them to remote repo.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (support)
$ touch mysupportfile1.txt

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (support)
$ git add .

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (support)
$ git commit -m "Third"
[support 38dc70f] Third
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 mysupportfile1.txt

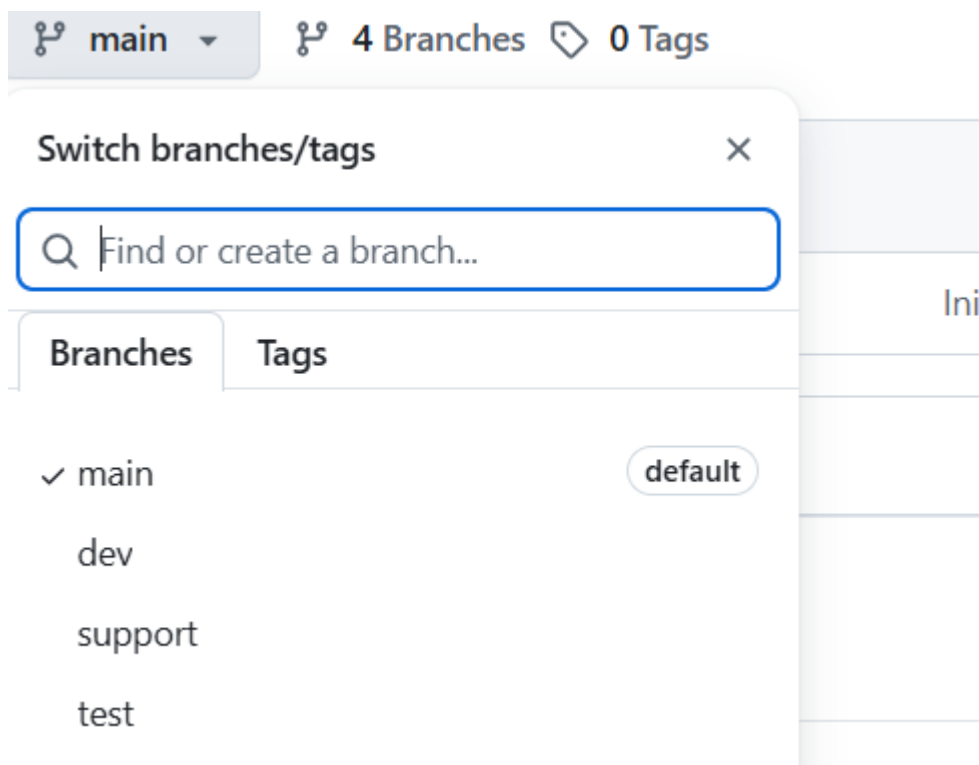
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (support)
$ git push origin support
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 251 bytes | 125.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'support' on GitHub by visiting:
remote:   https://github.com/NITHYAVEL04/myrepo1/pull/new/support
remote:
To https://github.com/NITHYAVEL04/myrepo1.git
 * [new branch]      support -> support
```

If we check the commit history from support, we can see the commits made in main, dev and test along with support.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (support)
$ git log --oneline
38dc70f (HEAD -> support, origin/support) Third
f4e8d52 (origin/test, test) Second
ad7e99a (origin/dev, dev) First
cf80d01 (origin/main, origin/HEAD, main) Initial commit
```

Step no:9 Now go to the git hub account we can see all the four branches

Refresh to see them.



To delete those branches: **git branch -D branch name**

Before deleting it , switch to main then only we can delete them.

```
nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git branch -D support
Deleted branch support (was 38dc70f).

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git branch -D test
Deleted branch test (was f4e8d52).

nithya@DESKTOP-NBCS2S9 MINGW64 ~/downloads/myrepo1 (main)
$ git branch -D dev
Deleted branch dev (was ad7e99a).
```